1.1 Каркас сайта. Модульная сетка

Любая веб-страница состоит из укрупненных структурных блоков, например, как на рисунке 60:

– Шапка (header);
– Основная часть (main);
– Боковые панели (sidebar);
– Подвал (footer).

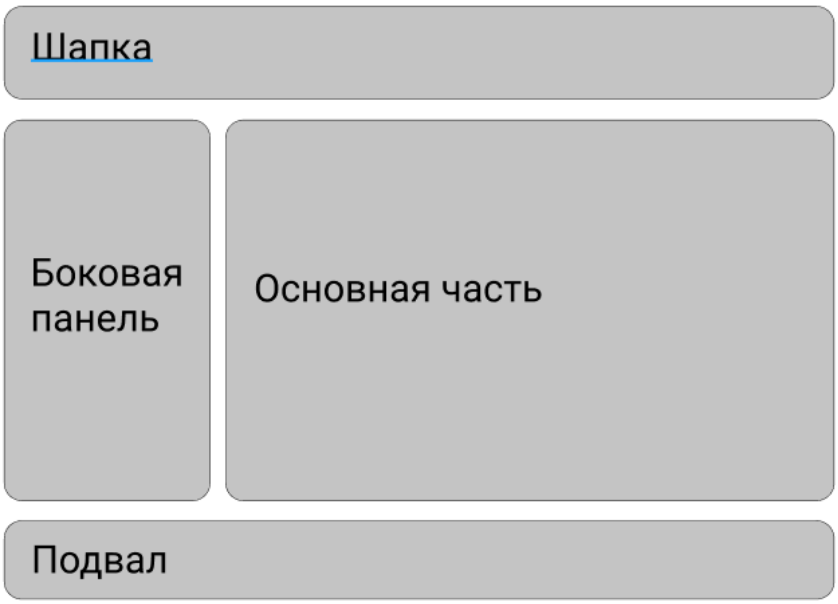

Рисунок 60 – Примерная структура страницы сайта

В свою очередь эти блоки делятся на более мелкие блоки. Внутри блоков располагаются различные элементы (текст, изображения, ссылки, видео и т.д.).

Модульная сетка (в веб-дизайне) представляет собой единую схему расположения всех блоков и элементов сайта. Модульная сетка представляет собой некоторый каркас, который проходит через всю страницу и определяет расположение всех блоков и элементов на ней.

В веб-дизайне чаще всего используется колоночная модульная сетка (рис. 61). Обычно сетка состоит из 12-ти колонок.

Рисунок 61 – Двенадцати-колоночная модульная сетка

При этом вся рабочая область страницы разбивается на 12 колонок и структурные блоки страницы имеют ширину, кратную числу колонок. Например, ширина в 1 колонку, в 2 колонки и т.д. до 12 колонок.

Колонки в веб-дизайне могут иметь конкретную абсолютную ширину, заданную в пикселях, а могут иметь ширину относительную, при этом ширина колонок меняется с изменением ширины окна браузера.

Существует три основных способа построения сеток:

1. Тег `div` с CSS-свойствами `display - block, inline, inline-block`.
2. Одномерные гибкие блоки `flexbox`.
3. Система двумерных сеток `grid layout`.

Первый способ был рассмотрен в практических работах первой главы пособия. Подробно рассмотрим второй способ построения сеток.

1.2 Flex-контейнеры

Одним из современных способов позиционирования блоков на странице является использование гибких блоков (flexbox). Для того чтобы простые блоки стали гибкими, достаточно у их **родителя** задать свойство `display:flex`.

Рассмотрим основные свойства, которые можно задавать у flex-контейнера (родительского элемента гибких блоков).

У flex-контейнера есть две оси – главная ось (по умолчанию направлена слева направо и поперечная ось – по умолчанию направлена сверху вниз). Блоки внутри flex-контейнера располагаются вдоль главной оси (рис. 62).

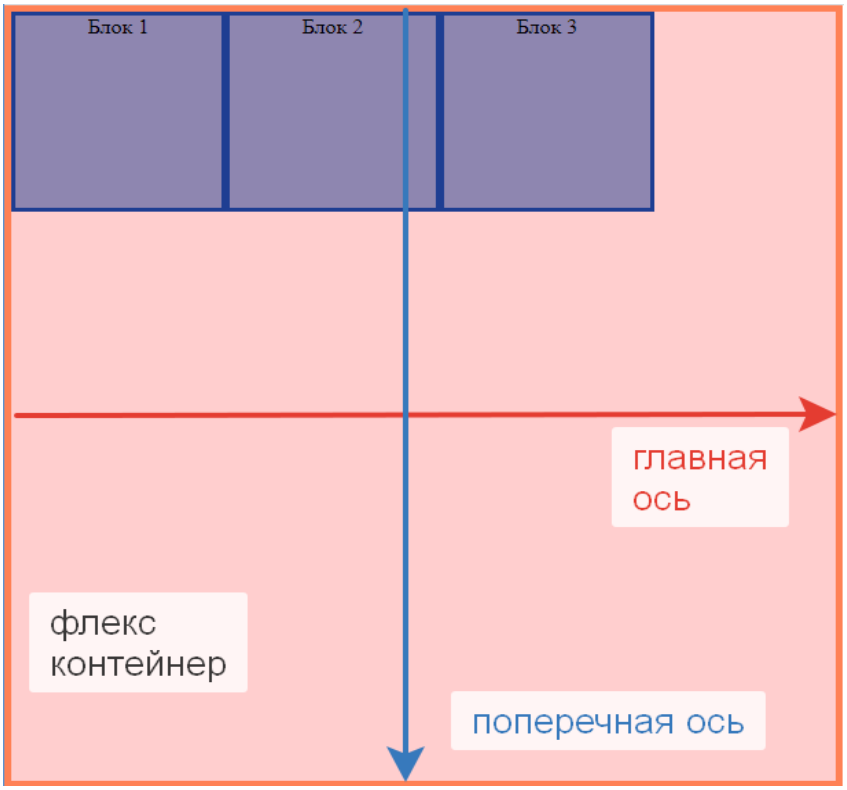

Рисунок 62 – Flex-контейнер (направление осей по умолчанию)

Изменить направление главной оси, а, следовательно, и изменить порядок вывода блоков внутри flex-контейнера, можно с помощью CSS-свойства **flex-direction.** Возможные значения:

- `flex-direction:row;` (по умолчанию слева направо);
- `flex-direction:column;` (сверху вниз, рис. 63);
- `flex-direction:row-reverse;` (справа налево);
- `flex-direction:column-reverse;` (снизу вверх).

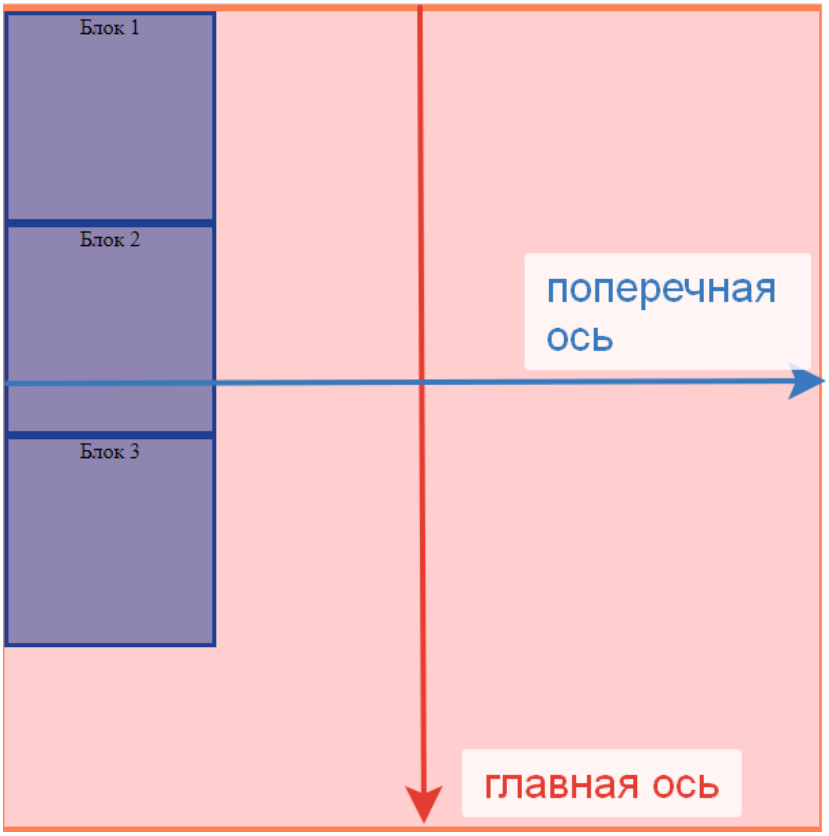

Рисунок 63 – Изменение направления главной оси

Поперечная ось используется при выравнивании элементов. Если главная ось направлена горизонтально, то поперечная ось смотрит вниз. Если главная ось направлена вертикально, то поперечная ось смотрит направо. Направление поперечной оси изменяется вместе с изменением главной оси. Поперечная ось не может быть направлена вверх или влево.

Выравнивание по главной оси задается с помощью CSS-свойства **justify-content.** Возможные значения (рис. 64):

- `flex-start`,
- `flex-end`,
- `center`,
- `space-between`,
- `space-around`.

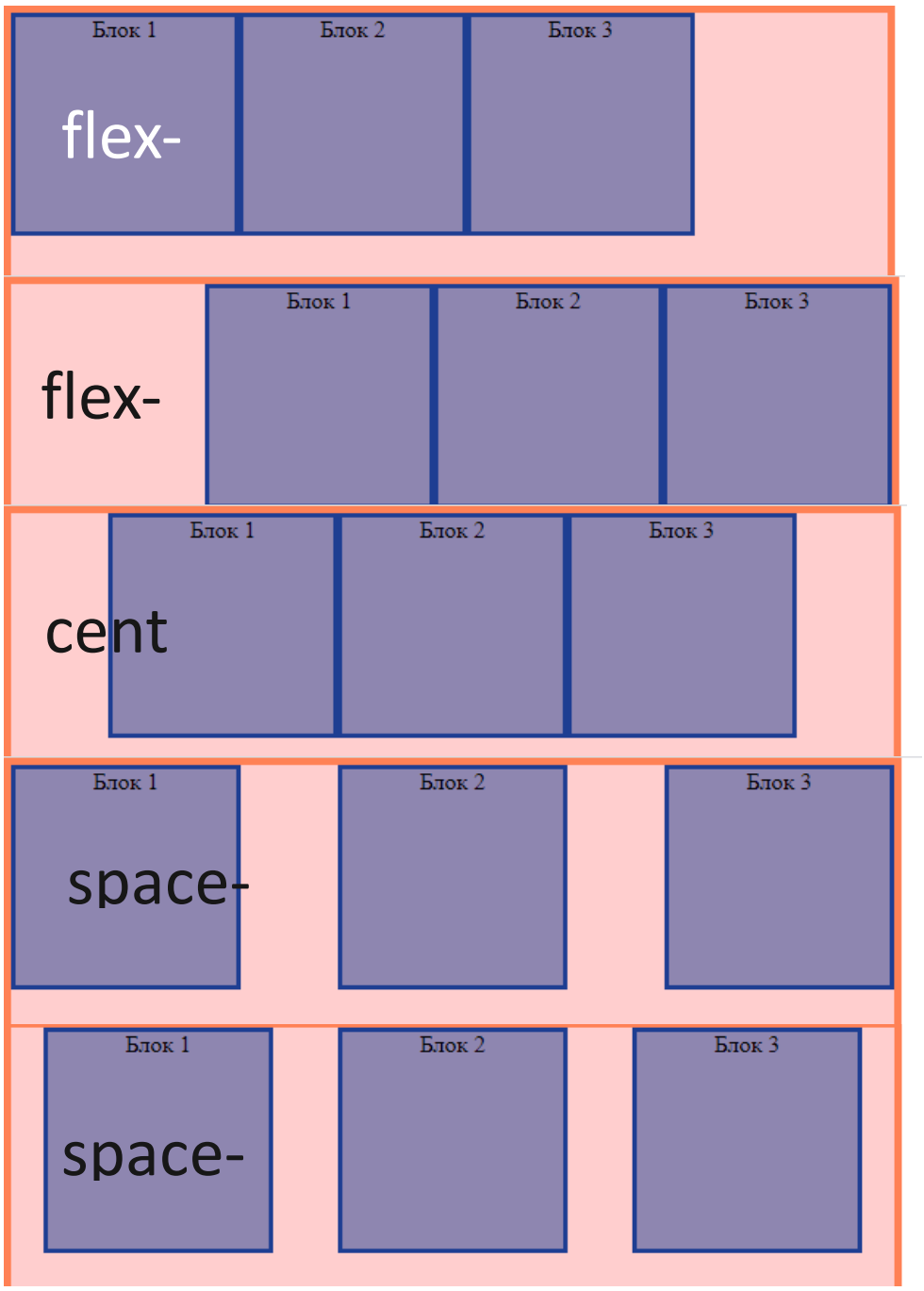

Рисунок 64 – Выравнивание по главной оси

Выравнивание вдоль поперечной оси осуществляется с помощью CSS-свойство **align-items**. Возможные значения:

– `stretch` (значение по умолчанию, если у flex-элементов не задана высота, то при этом свойстве элементы приобретают высоту родителя – растягиваются (рис. 65), если высота задана, то элементы выравнивается в начале поперечной оси);

– `flex-start` (выравнивание элементов в начале поперечной оси);

– `flex-end` (выравнивание элементов в конце поперечной оси);

– `center` (выравнивание элементов по центру поперечной оси);

– `baseline` (выравнивание элементов по базовой линии flex-элементов – рисунок 66).

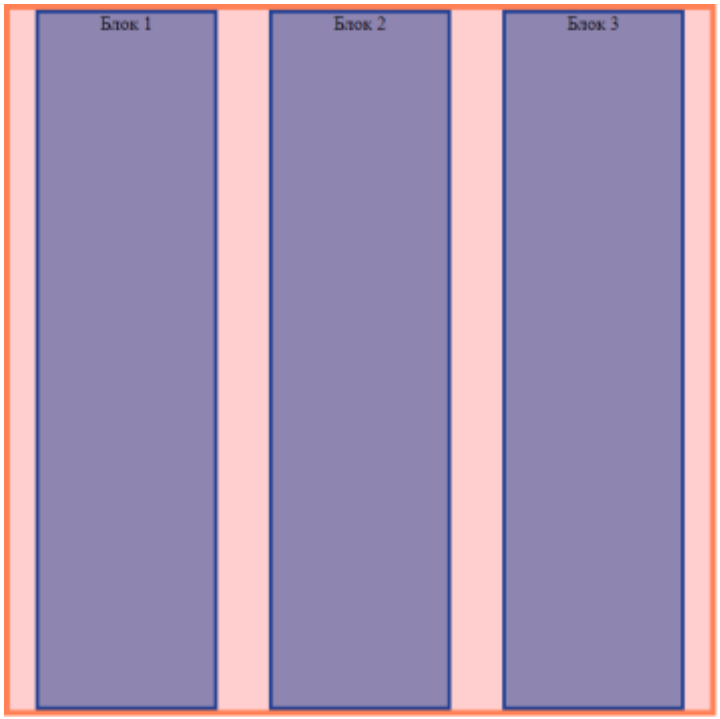

Рисунок 65 – Значение выравнивания `stretch` (у блоков не задана высота)

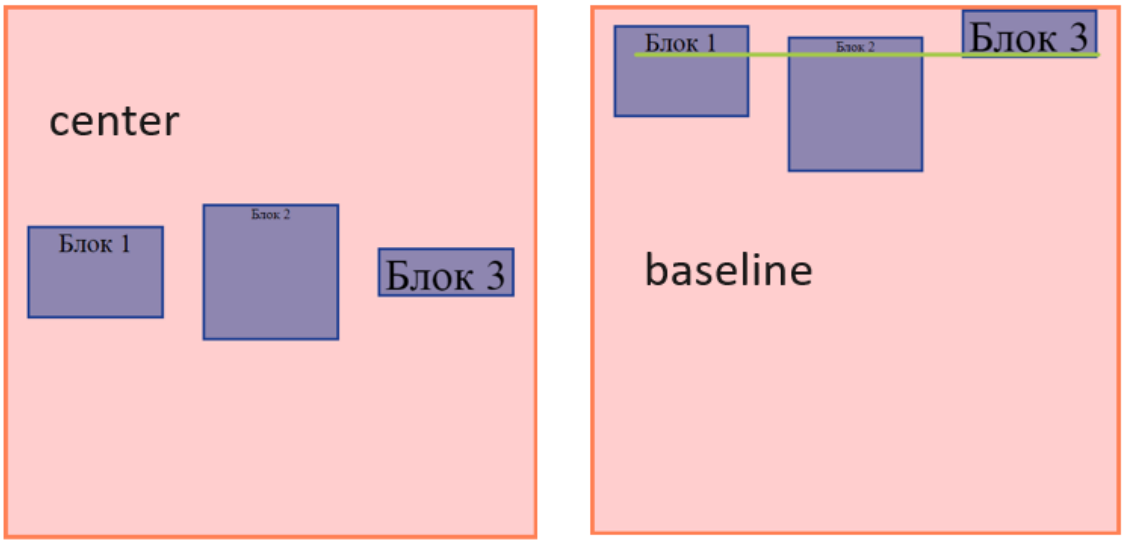

Рисунок 66 – Выравнивание по центру (слева) и выравнивание по базовой линии (справа)

Следующее свойство связано с переносом элементов. Сначала определим поведение элементов внутри flex-контейнера. Если суммарная ширина элементов меньше ширины контейнера, то они располагаются внутри контейнера, не меняя своей ширины (рис. 67).

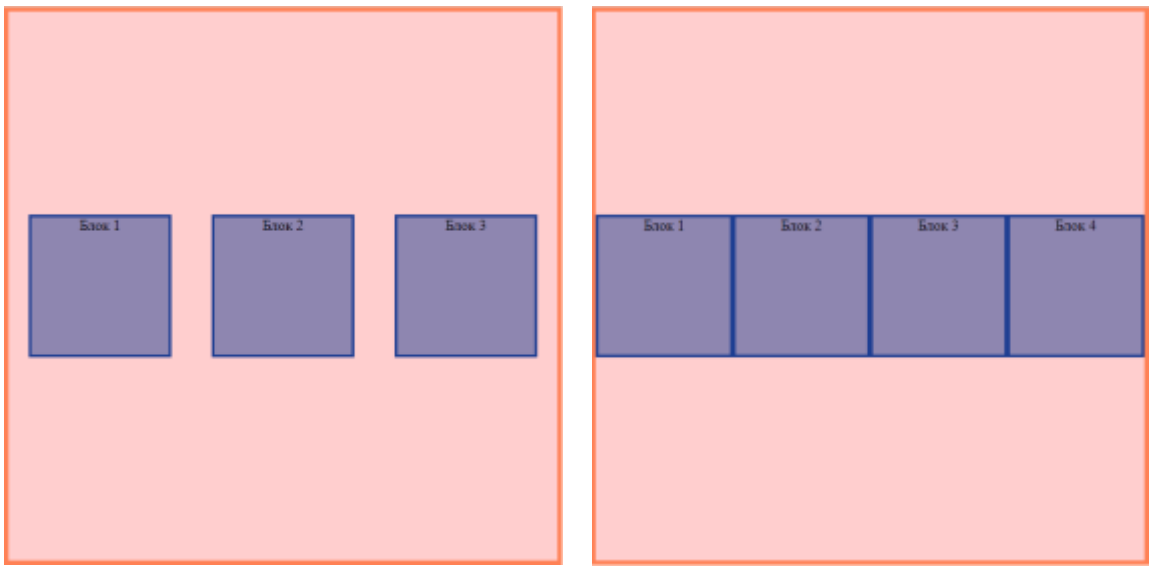

Рисунок 67 – Расположение элементов внутри контейнера

Если суммарная ширина элементов больше ширины родителя, то элементы сужаются (рис.68).

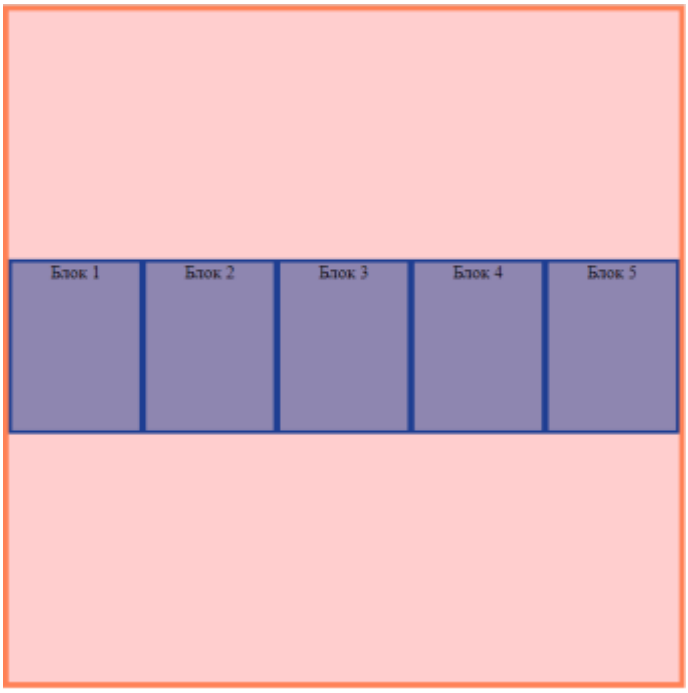

Рисунок 68 – Расположение элементов внутри контейнера (сужение ширины элементов)

Если содержимое элементов не позволяет им сузиться под ширину родителя, то происходит переполнение контейнера и элементы выходят за его пределы (рис 69).

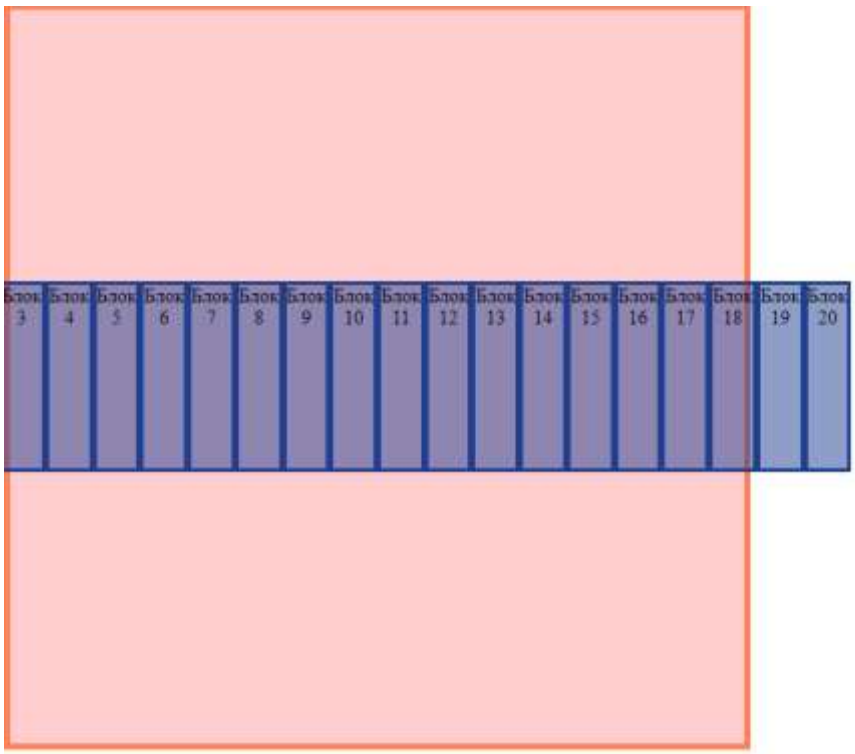

Рисунок 69 – Расположение элементов внутри контейнера (переполнение)

Поведение элементов при переполнении можно поменять с помощью CSS-свойства **flex-wrap.** Возможные значения

– `nowrap` – перенос flex-элементов на новую строку запрещён (значение по умолчанию).

– `wrap` – разрешает перенос flex-элементов на новую строку. Ряды элементов располагаются вдоль поперечной оси, первый ряд – в начале поперечной оси, последний – в конце (рис. 70).

– `wrap-reverse` – разрешает перенос flex-элементов на новую строку. Ряды элементов располагаются в обратном порядке: первый – в конце поперечной оси, последний – в начале (рис. 71).

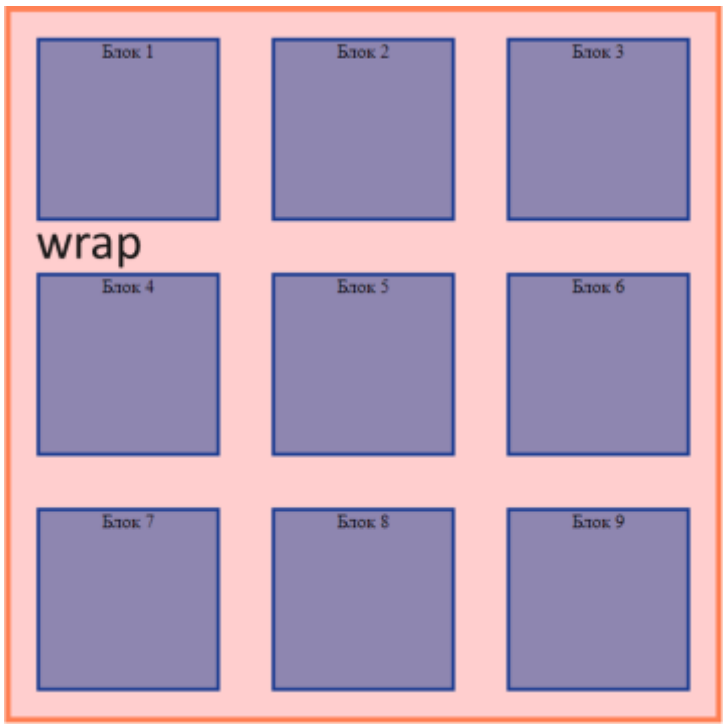

Рисунок 70 – Перенос элементов (`flex-wrap:wrap`)

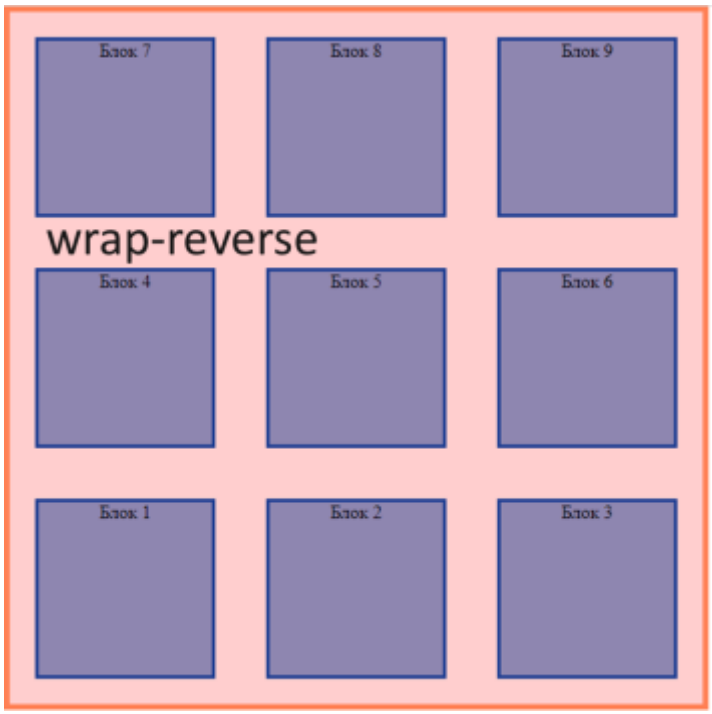

Рисунок 71 – Перенос элементов (`flex-wrap:wrap-reverse`)

Для выравнивания вдоль поперечной оси нескольких строк блоков используется свойство CSS-свойство **align-content**. Возможные значения свойства:

– `stretch` – значение по умолчанию, растягивает ряды flex-элементов, при этом оставшееся свободное место между ними делится поровну. Отображение строк при этом зависит от значения свойства `align-items`:

- если у `align-items` задано значение `stretch`, то элементы в строках растягиваются на всю высоту своей строки.
- если значение отлично от `stretch`, то элементы в строках ужимаются под своё содержимое и выравниваются в строках в зависимости от значения `align-items`.

– `flex-start`,
– `flex-end`,
– `center`,
– `space-between`,
– `space-around`.

Свойство `align-content` переопределяет заданное значение `align-items`.

Flexbox по своей сути является моделью одномерного макета. Flex-элементы внутри flex-контейнера можно размещать как горизонтально, так и вертикально, но не одновременно. Если вы хотите располагать элементы в двух направлениях, вам потребуется вставить один flex-контейнер в другой (рис. 72).

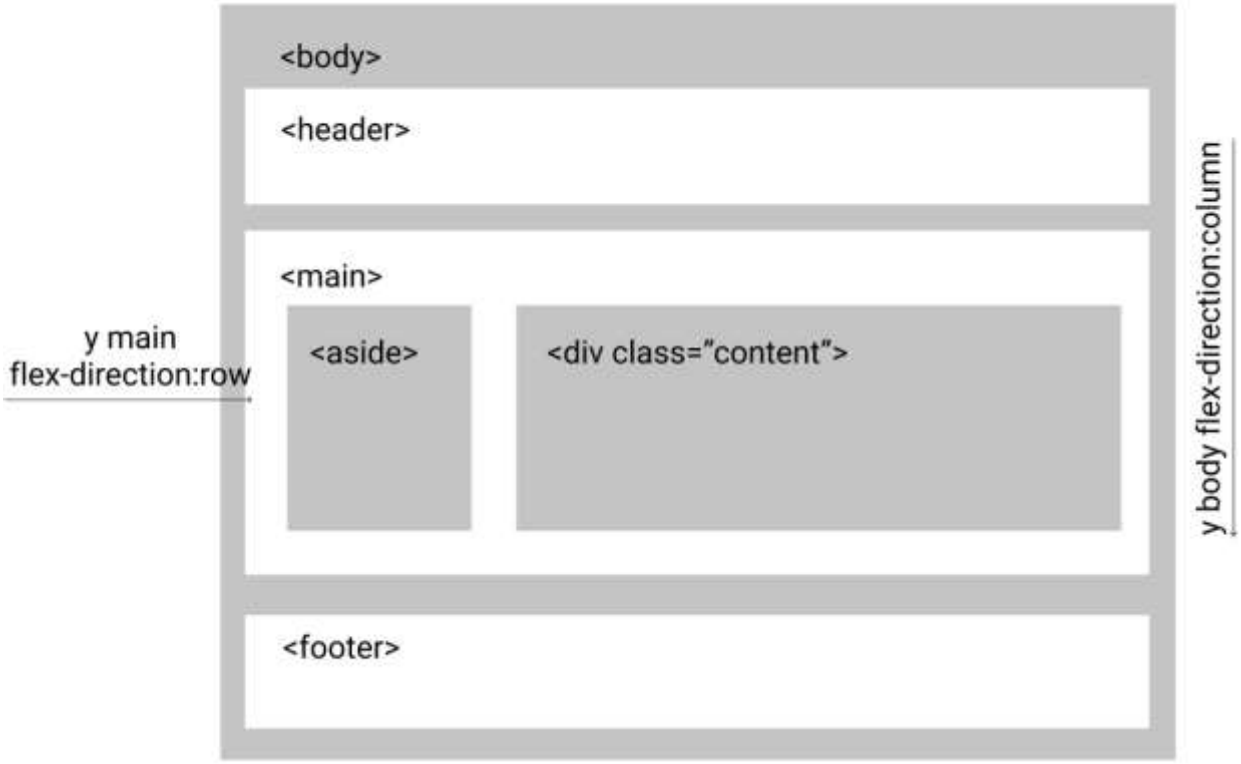

Рисунок 72 – Размещение flexbox

1.3 Flex-элементы

Flex-элементы имеют ряд отличий от обычных блочных элементов:

– flex-элементы не растягиваются на всю ширину контейнера по умолчанию;

– на flex-элементы не действует свойство float;

– внешние отступы не схлопываются ни по горизонтали, ни по вертикали;

– внешние отступы не выпадают ни из flex-контейнера, ни из flex-элементов.

При этом свойства `border` и `padding` у flex-элементов работают также, как и у блочных. Но `margin`, как и `padding` «не знают» понятия направление оси. Поэтому, когда главная ось направлена слева направо, горизонтальные отступы перемещают flex-элементы вдоль главной оси. Но если направить главную ось сверху вниз, то те же отступы начнут работать вдоль поперечной оси. Аналогично работают стили для высоты и ширины блоков:

– ширина width – размер блока по горизонтали всегда,

– высота height – размер блока по вертикали всегда.

У flex-элементов есть собственное свойство, которое задает размер элемента по главной оси – **flex-basis**. В зависимости от того, как направлена ось, это свойство может задавать ширину или высоту блока.

Следующее свойство отвечает за распределение свободного места между элементами внутри контейнера. Механизм распределения свободного места при использовании flex:

– находятся элементы, у которых есть внешние отступы со значением auto;

– всё свободное место в соответствующих направлениях отдаётся таким отступам (то есть задаётся определённый размер отступа в пикселях);

– если элементов с автоматическими отступами на одном направлении несколько, то место между ними перераспределяется поровну;

– только после этого запускаются механизмы выравнивания.

Рассмотрим важные свойства flex-элементов. Первое свойство – это порядок вывода элементов на страницу. По умолчанию у всех элементов на странице порядковый номер равен 0, и они выводятся в том же порядке, в каком расположены в коде.

CSS-свойство **order** меняет порядковый номер элемента. В качестве значения можно назначить положительное или отрицательное целое число. Сортировка производится по возрастанию номера. Таким образом отрицательное число для свойства `order` смещает элемент вперед, а положительное – в конец.

CSS-свойство, отвечающее за растягивание блока – **flex-grow** принимает неотрицательные числовые значения, его значение по умолчанию – 0. Если

значение `flex-grow` равно 0, то flex-элемент «не претендует» на оставшееся свободное место во flex-контейнере и не будет увеличиваться, чтобы занять это место. Если значение `flex-grow` больше нуля, то flex-элемент будет увеличиваться, «захватывая» оставшееся свободное место.

CSS-свойство **flex-shrink** принимает неотрицательные числовые значения, его значение по умолчанию – 1. Если значение `flex-shrink` больше нуля, то flex-элемент будет уменьшаться, «впитывая» часть отрицательного пространства, если оно существует. Если значение `flex-shrink` равно нулю, то flex-элемент уменьшаться не будет.

Подробный расчёт коэффициентов сжатия и растяжения можно посмотреть в источнике [6].